



Blazor Essencial

Parâmetros em Cascata





Parâmetros em Cascata

Abordagem tradicional com parâmetros : **Parameter e EventCallback**



Abordagem com parâmetros em cascata : **CascadingValue com CascadingParameter**





Parâmetros em cascata : CascadingValue com [CascadingParameter]

Os parâmetros em cascata permitem **compartilhar dados** através de uma hierarquia de componentes sem a necessidade de repassar explicitamente por cada nível

Isso é útil para informações globais :

- Tema de uma interface , estilos, etc.(ex:dark/light),
- Usuário autenticado
- Configurações globais da aplicação

<CascadingValue>: Componente que 'publica' ou 'fornece' um valor para todos os seus componentes **descendentes**. Tudo o que estiver dentro de um **<CascadingValue>** pode ter acesso ao valor que ele está *cascateando*.

[CascadingParameter]: Atributo usado em uma propriedade dentro de um componente filho. Ele '**consume**' ou '**recebe**' o valor que foi fornecido pelo **<CascadingValue>** mais próximo na hierarquia.

@ Exemplo : CascadingValue com [CascadingParameter]

ComponentePai

```
@page "/estilo-cascading"
<h3 style="@Estilo">Componente Pai</h3>
<CascadingValue Value="@Estilo">
    <ComponenteFilho/>
</CascadingValue>

@code {
    public string Estilo { get; set; } = "color:blue;font-weight: bold";
}
```

Usamos o **CascadingValue** que fornece o valor que será passado para o componente filho e que **envolve o componente filho** que vai receber o valor

ComponenteFilho

```
<h1 style="@EstiloRecebido"> Componente Filho com estilo do pai</h1>
@code {
    [CascadingParameter]
    public string EstiloRecebido { get; set; }
}
```

Define um parâmetro do tipo **CascadingParameter** do mesmo tipo declarado no componente pai e recebe o valor e o usa para aplicar o estilo

@ Exemplo com mais de um parâmetro (tipos diferentes)

```
@page "/estilo-cascading"
<h3 style="@Estilo">Componente Pai</h3>
<CascadingValue Value="@Estilo">
    <CascadingValue Value="@Data">
        <ComponenteFilho> </ComponenteFilho>
    </CascadingValue>
</CascadingValue>
@code {
    public string Estilo { get; set; } = "color:blue;font-weight:bold";
    public DateTime Data { get; set; } = DateTime.Now();
}
```

ComponentePai

Usamos dois **CascadingValue** para definir os valores **Estilo** e **Data** que desejamos passar para o componente filho

```
<h3 style="@EstiloRecebido">Componente Filho com estilo do pai </h3>
<h2>Data recebida do pai : @DataRecebida.ToShortDateString()</h2>
@code {
    [CascadingParameter] public string EstiloRecebido { get; set; }
    [CascadingParameter] public DateTime DataRecebida { get; set; }
}
```

ComponenteFilho

Define dois parâmetros do mesmo tipo definido no componente pai com **CascadingParameter** para receber os valores



Exemplo com parâmetros do mesmo tipo usando a propriedade **Name**

```
<h3 style="@Estilo">Componente Pai</h3>
<CascadingValue Value="@Estilo" Name="Estilo">
  ...
  <CascadingValue Value="@Fonte" Name="Fonte">
    <ComponenteFilho> </ComponenteFilho>
  ...
</CascadingValue>
@code {
  public string Estilo { get; set; } = "color:blue; font-weight: bold;";
  public string Fonte { get; set; } = "font-family: 'Courier New'; font-size: 16px;";
}
```

ComponentePai

Usamos dois **CascadingValue** para fornecer os valores e **Estilo** e **Data** usando a propriedade **Name** para identificar cada parâmetro

```
<h3 style="@EstiloRecebido">Componente Filho </h3>
<p style="@FonteRecebida">Texto com estilo herdado do pai</p>
```

ComponenteFilho

```
@code {
  [CascadingParameter(Name= "Estilo")] public string EstiloRecebido { get; set; }
  [CascadingParameter] public DateTime DataRecebida { get; set; }
  [CascadingParameter(Name="Fonte")] public string FonteRecebida { get; set; }
}
```

Define dois parâmetros do mesmo tipo definido no componente pai usando **CascadingParameter** com a propriedade **Name** com os nomes dos parâmetros recebidos



Atualização dinâmica de Valores em Cascata : IsFixed

Por padrão, quando o valor de um **CascadingValue** muda, todos os componentes que o consomem são automaticamente renderizados novamente. Isso é controlado pelo parâmetro **IsFixed**.

Se sabemos que um valor *não mudará durante a vida útil* do componente, podemos usar **IsFixed="true"** para otimizar a performance, indicando ao Blazor que não precisa monitorar alterações.

Se o valor for **'true'**, o valor é **fixo** e não muda após a inicialização, evitando assinaturas desnecessárias nos descendentes

```
<CascadingValue Value="versao" IsFixed="true">  
  <ComponenteFilho />  
</CascadingValue>
```

```
@code {  
    public string AppVersion { get; set; } = "1.0.0";  
}
```

Com **IsFixed='true'**, mudanças no **AppVersion** não propagam, o que economiza recursos.

@ Usando CascadingValue<T>

A partir do .NET 9, temos uma sintaxe mais forte para trabalhar com parâmetros em cascata usando **CascadingValue<TValue>**.

O **CascadingValue<TValue>** é uma variação do componente **CascadingValue** que permite especificar o **tipo de dado** que está sendo passado para a **hierarquia** de componentes.

O **TValue na notação <TValue>** é um *placeholder* para o tipo de dado real que você deseja compartilhar, como uma *classe*, *um record*, *um struct* ou *uma interface*.

```
<CascadingValue Value="@usuario" TValue="Usuario">  
  <CascataUsuarioFilho />  
</CascadingValue>
```

Pai

```
@code {  
    private Usuario usuario = new("João", true);  
  
    public record Usuario(string Nome, bool Autenticado);  
}
```

```
@if (UsuarioAtual?.Autenticado == true)  
{  
    <p>Bem-vindo, @UsuarioAtual.Nome!</p>  
}
```

Filho

```
@code {  
    [CascadingParameter]  
    public Usuario? UsuarioAtual { get; set; }  
}
```


@ Uso de parâmetros

Use **[Parameter]** para comunicação **pai-filho** direta

Use **EventCallback** para o filho **notificar** pai

Parâmetros em cascata com **CascadingValue** e **[CascadingParameter]** são para dados hierárquicos

Apenas **um valor em cascata com o mesmo nome** pode estar **ativo** em uma **hierarquia** – o Blazor usa o valor mais próximo na árvore.

Mudanças em **cascata** acionam **OnParametersSetAsync** nos filhos, integrando com o ciclo de vida.(*)

Eles não cruzam fronteiras de **modos de renderização**, como de **SSR estático** para **interativo**, porque os valores precisam ser **serializáveis em JSON** para transições assim, e nem sempre são.